Reproducing *Real-Time Detection and Repair of LLM Agent Failures*

A controlled reproduction of the behavioural-monitor table, and one ablation of its fusion rule

Name: Sun Yusu Student ID: 1155267751 FTEC5660 Reproducibility Work

Paper: arXiv:2608.02464 (S. Dubey, 2026)
Upstream: `github.com/sunnydubey1111/agent-trajectory-sentinel`
Our fork: `github.com/SamielSun/Individual-project-trajectory-sentinel-reproduce`
Recorded presentation: `slides/presentation.mp4` in the repository (8:48)

---

1 Project summary and what we tried to reproduce

AgentTrajectorySentinel watches a running agent episode and raises an alarm at derailment onset, a few steps before the task fails, using only the telemetry each step emits. The monitor is a one-class echo-state-network ensemble with a CUSUM per channel, fitted on healthy episodes only. A second layer checks results deterministically: it recomputes a run's stated total from the tool results that run actually received. When either layer fires, the run is rolled back to a checkpoint and re-run, which is the repair arm. The project is a monitoring and repair layer over agent trajectories, listed in the course's Section 8.1 as Tier A. Our object of study is the monitored episodes.

We reproduce one table from the paper, its "behavioural monitor, synthetic testbed, five seeds" table, and one of the mechanism claims the table is used to support:

> "the per-channel max-fusion wrapper, not the reservoir, carries most of the margin"

Following Section 2 of the assignment, that means one row set on one benchmark (the in-process synthetic testbed) at a 5% healthy-validation false-alarm budget, over the paper's five seeds. Recomputed from the committed table, the primary row is detection 0.707 ± 0.068, steps saved 4.61 ± 0.97 and episode AUC 0.872 ± 0.015; the paper prints those to fewer digits, as 0.71 ± 0.07, 4.6 ± 1.0 and 0.872 ± 0.015, and the comparator rows are reproduced with it.

The paper states its numbers are bit-reproducible from a single master seed, and ships a claim ledger that recomputes every headline figure from committed tables. Most of the work is therefore in deciding what exactly is reproduced and what silently changes the answer, which is what Sections 4 and 7 take in turn.

2 Setup notes

The repository is a clone of upstream at commit `1b3e07f` with no source edits. Every experiment goes through a thin overlay (`repro/`). The overlay runs each arm in a fresh subprocess, redirects the upstream output directory through the documented `AGENTWATCH_RESULTS_ROOT` hook so the committed `results/` tree is never overwritten, and applies the modification of Section 5 by monkeypatch. The working tree stays byte-clean, which keeps the `git_dirty` provenance flag in `results.json` meaningful.

The machine is Windows 11 with 22 logical cores, CPU only, no GPU, no Docker and no database. The interpreter is Python 3.12.9 with numpy 2.4.6, scipy 1.17.1, pandas 3.0.3 and scikit-learn 1.9.0. The paper's artifacts were produced on CPython 3.14.5; the table below says what that changes. Input data is the simulator in `derail/telemetry/generator.py` plus the committed `results/` tree, and nothing is downloaded during a run. No credential is read or stored; `.env` is gitignored. One arm takes 155 s of wall clock and the study is 13 arms.

Every arm is CPU-bound and seeded. Wall-clock time is the only quantity that varies between runs, which is why the tables below compare columns rather than distributions.

Table 1: Deviations from the paper's recorded environment, and what each one could change.

Item	Paper	What we did, and what it risks
Interpreter	CPython 3.14.5	3.12.9, because the 3.14 installer would not download at usable speed. All five package versions are the pinned ones and they ship `cp312` wheels. The risk is numeric drift if a library changed a default between minor versions. Section 4 shows the master-seed table reproduced to the last digit, so on this study there is none.
torch baselines	`torch==2.12.0+cpu`	not installed. The GRU/LSTM/TCN rows are therefore missing from our five-seed table. We do not report those rows and make no claim about them.
Repair arm	live model calls	not run. It needs a served local model (Ollama), and the paper itself marks those numbers as not bit-reproducible. Out of scope by Section 2.
External corpora	AFTraj-2K, ATBench	not run. Both need a download from HuggingFace and are scored on corpora the repository does not redistribute. Out of scope.

With the episode corpora present (`traces/`, fetched separately because this fork drops them), `pytest -m "not network and not ollama"` gives 854 passed, 14 failed and 21 skipped. All fourteen failures are missing optional dependencies of the trace-collection stack (`sqlalchemy`, `langchain-core`, `autogen-core`, `huggingface-hub`, `pyarrow`), which `requirements-core.lock.txt` excludes on purpose. None of them touches a number quoted here.

3 Reproduction targets and metric definitions

The table is built from four quantities, all computed on the held-out *test* split, with thresholds picked on a *healthy validation* split and monitors fitted on *healthy train* only. We take the definitions from the upstream evaluation module instead of re-deriving them:

- **detection rate**: the fraction of injected-failure test episodes whose first alarm lands after onset τ and before the episode ends. It uses only what was observable by the alarm step.
- **steps saved** (the paper's `mean_lead_all`): the expected budget saved per failure episode, counting a miss as 0 instead of dropping it. This is the survivorship-free version of how early the alarm arrived.
- **episode AUC**: an *offline* ranking statistic, each episode's maximum score against the healthy population. The paper warns that it cannot be earned online and that it rises with episode length. We report it as a ranking statistic and never as detection.
- **realised false-alarm rate**: the fraction of healthy test episodes that alarm, at a threshold chosen to hit a 5% budget. The budget is asked for on validation while the realised rate is measured, and the paper says the two do not always agree, so we keep both.

Aggregation over seeds follows the authors' rule: mean and *sample* standard deviation (ddof = 1) over the five seeds {20260713, 7, 101, 202, 303}. Upstream's `run_multiseed` hard-codes its output base, so we re-implemented the aggregation in `repro/collect.py` with the same monitor list, the same metrics and the same `ddof=1`, and compare the result against the committed `results/tables/multiseed_summary.csv`.

Table 2: Target values, read from the committed artifact (nothing is transcribed from the PDF).

Monitor	detection	steps saved	episode AUC	realised FA
`esn_cusum_max` (primary)	0.707 ± 0.068	4.61 ± 0.97	0.872 ± 0.015	0.044 ± 0.016

4 Results: our numbers against the paper's

4.1 Single seed

Running the unchanged pipeline at the master seed regenerates the paper's own master-seed artifact. Table 3 gives, for every numeric column of `results/tables/h1_main.csv`, the largest absolute difference between our file and the committed one, over the 19 monitor rows both files contain. Every reported metric agrees exactly. The single non-zero entry is `theta`, at 5.2e-11, a floating-point difference in a threshold quantile. It is far too small to move an alarm, and no alarm moved: detection, false-alarm rate, steps saved and AUC are all exactly zero.

Table 3: Master-seed reproduction: largest |ours − committed| per column of `h1_main.csv`. Generated by `repro/collect.py`.

Column of `h1_main.csv`	max |ours − committed|
`detection_rate`	0
`mean_lead_all`	0
`episode_auc`	0
`healthy_fa_rate`	0
`theta`	$5.2e-11$
remaining 11 numeric columns	0
all 16 numeric columns, 19 monitor rows	$5.2e-11$

4.2 Five seeds

Table 4 re-derives the paper's headline rows from our own five runs. The primary monitor lands at detection 0.707 ± 0.068, steps saved 4.61 ± 0.97 and AUC 0.872 ± 0.015, against the paper's 0.707 ± 0.068, 4.61 ± 0.97 and 0.872 ± 0.015: a difference of +0.0000 on detection and +0.0000 on AUC. The eight monitor rows we can run all match at full precision; the GRU/LSTM/TCN rows are absent, and Section 2 says why.

Table 4: Five-seed table (mean ± sample std, `ddof`=1), our re-run against the committed artifact.

	detection	steps saved	episode AUC
our re-run (5 seeds)	0.707 ± 0.068	4.61 ± 0.97	0.872 ± 0.015
committed artifact	0.707 ± 0.068	4.61 ± 0.97	0.872 ± 0.015

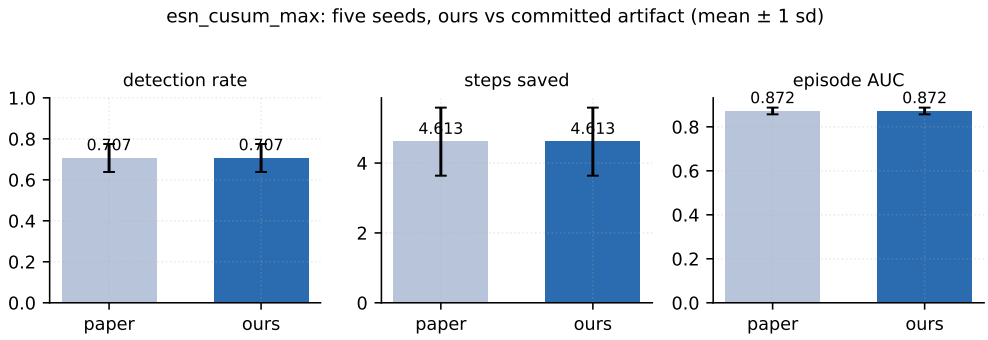


Figure 1: Five-seed headline metrics for the primary monitor. Error bars are ±1 sample std over the five seeds.

4.3 What did not reproduce

Three gaps.

Wall-clock cost, 219 μs per step, was not attempted. The paper calls it the one figure that measures the machine and reports it moving to 252 μs on the same box, so we make no latency claim.

The GRU/LSTM/TCN rows are absent because torch is an optional import behind a `try/except`. The upstream design makes that gap silent: the monitor list shrinks and the table still renders. We list it here so that a reader comparing row counts knows why.

The corpus size changed. The arXiv v1 text describes 2,823 episodes across 25 corpora at commit `00c0673`, while the tree we pinned carries 4,022 across 35. A table quoted without the commit can end up compared against the wrong denominator, so we pinned ours and record it in `out/environment.json`.

5 Our modification: replacing max fusion with mean fusion

The modification touches one thing: how the three per-channel CUSUM streams of the primary monitor are combined. Upstream, `ChannelMaxESNMonitor.score_step` returns the maximum over the channels $\{e, u, m\}$. We use the arithmetic mean, and the mean of the top two streams as a graded control. The three sub-monitors, their reservoir weights and seeds, the CUSUM, the standardiser, the split discipline and the 5% threshold rule all stay fixed. The patch applies at run time and touches only the monitor named `esn_cusum_max`; the other channel variants keep `max` and serve as untouched controls.

Max fusion is supposed to matter because a shift confined to one channel survives a maximum and is diluted by a mean, and the paper's example is grounding loss, which lives in 4 of 43 uncertainty dimensions. That makes the claim falsifiable in one direction: if the max carries the margin, removing it has to cost a large part of it.

The three arms run on the same two disposable seeds (`424242`, `31337`), which the published study never uses. Same code, same machine, same pinned environment; only the aggregation function differs.

Table 5: Fusion ablation on the primary monitor, paired over two unseen seeds and averaged over the two. Control comparisons: 18 monitors at each seed, 36 in total, all bit-identical under the patch.

Aggregation	detection	steps saved	episode AUC	realised FA
max (baseline)	0.7025	4.33	0.8832	0.0250
mean	0.7137	5.00	0.8786	0.0375
Δ against max	+0.0113	+0.67	-0.0046	+0.0125
top-2 mean	0.7112	4.70	0.8804	0.0250

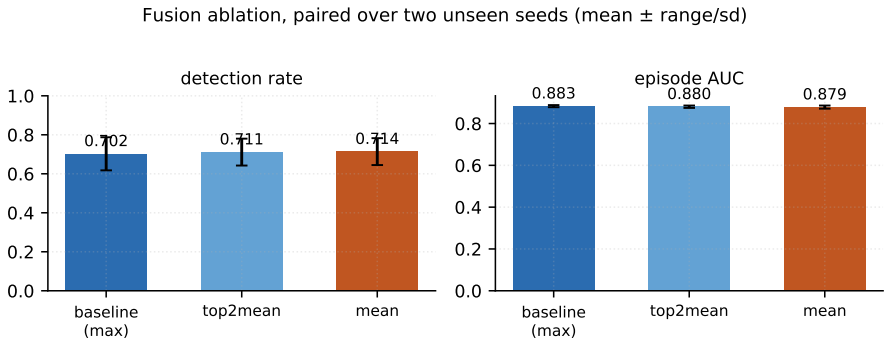


Figure 2: Fusion ablation: detection and episode AUC by aggregation rule, paired over the two unseen seeds.

The predicted collapse does not happen. Replacing max with a mean moves detection from 0.7025 to 0.7137 (+0.0113), adds +0.67 steps saved and gives up −0.0046 of ranking AUC. The realised healthy false-alarm rate at the same nominal 5% budget rises from 0.0250 to 0.0375 (+0.0125), while

the top-2 control, which behaves like max here, sits at 0.0250. All 36 control comparisons stay identical, so the patch is isolated.

Three things keep this short of a clean result, and they matter at the size of the effect. The detection gain comes from one seed: 0.6425 to 0.6650 on `424242`, and 0.7625 to 0.7625 on `31337`. Because the mean arm also alarms more often, the detection numbers are not compared at a matched false-alarm rate, so part of the gain could be the looser operating point rather than better fusion. And two seeds give no interval, so what follows is a direction.

With that caveat, the ablation argues against the paper's attribution. Removing the max costs nothing in detection and costs about a percentage point of false-alarm rate, which is the opposite of the ordering the claim implies.

6 Debug diary

1. Re-running the study would have destroyed the thing it is compared against. The study writes into `results/`, which is exactly where the paper's committed tables live, so the first run would have overwritten the reference numbers. Upstream exposes `AGENTWATCH_RESULTS_ROOT` for sensitivity runs; every arm is redirected through it, and ours and theirs stay two independent artifacts.
2. The five-seed summary could not be reused as it ships. `run_multiseed` hard-codes `results/` as its base and only forwards `--seed`, so once the arms are redirected it looks for results that are not there. I re-implemented the aggregation in `repro/collect.py` with the authors' rule (mean and sample standard deviation, `ddof=1`) and the same monitor list, then checked the output against the committed `multiseed_summary.csv`: every row matched, which is also the check that my re-implementation was faithful.
3. The upstream test gate expects corpora this fork does not ship. A sparse checkout without `traces/` left 62 tests failing on a missing `manifest.json`. Fetching the corpora brought the gate to 854 passed, 14 failed and 21 skipped, and all fourteen remaining failures are optional dependencies of the trace-collection stack. A fork cannot inherit a test gate without inheriting the data behind it.
4. Silent row loss. Without torch the monitor list loses the GRU, LSTM and TCN rows and the table still renders, with no error anywhere. A reader comparing row counts would see three missing rows and no explanation. The fix is to assert on the row set instead of trusting that a file exists, which is what `repro/ci_gate.py` now does.
5. I over-claimed the coverage in a draft of this report. An earlier version said all eleven monitor rows of the five-seed summary matched. Counting the columns showed that eleven rows exist but only eight carry values on our side; the other three are the torch baselines. The claim is now eight rows plus a stated gap, and the same correction went into the README and the deck.
6. "Exact" needed a definition before it could be claimed. Every reported metric reproduces to zero, but the alarm threshold θ differs by 5.2×10^{-11}, because it is a float quantile of the healthy scores. Reporting the table as exact therefore required deciding what has to be zero: the metrics, not the threshold. Since detection, false-alarm rate, steps saved and AUC are all zero, no alarm changed side, so the distinction can be checked instead of argued.

7 Conclusions

- **Fully reproducible:** the behavioural-monitor table. At the master seed every reported metric in `h1_main.csv` matches the committed file, with the alarm threshold differing by 5.2×10^{-11}; over five seeds every monitor row we can run matches, including the primary monitor at detection 0.707 $\pm$ 0.068 against the committed 0.707 $\pm$ 0.068. The reproduction is deterministic and CPU-only.
- **Contradicted by our ablation, weakly:** the attribution of the margin to the max rule. Replacing max with a mean leaves detection at 0.7137 (from 0.7025) and costs -0.0046 AUC,

while the realised false-alarm rate rises +0.0125. The gain comes from one of two seeds and the arms are not compared at a matched false-alarm rate, so this is a direction rather than a measured effect. All 36 control comparisons are unchanged.

- **Not reproduced:** wall-clock latency, the GRU/LSTM/TCN rows, the live repair arm and the two external corpora. Each is a coverage gap declared in Section 2 of this report; none of them is a defect in the code.
- **Hidden assumptions.** The paper's v1 corpus and the current tree differ in size, so quote the commit. `episode_auc` is an offline ranking statistic, and reporting it as detection inflates the result. The realised false-alarm rate is not the budget. An optional dependency can change the row set without an error.
- **What a rerun would need.** Pin the commit, redirect the results root, assert on the row set and, for the ablation, enough seeds to put an interval on the effect.

Every number this reproduction produces comes from the harness in the repository: `python -m repro.env`, `python -m repro.run_all`, `python -m repro.collect` and `python -m repro.figures` regenerate it from scratch (13 arms, about 155 s each).